IBM Innovation Centre for Education

NCS Nihon Communication Solutions

YENEPOYA (DEEMED TO BE UNIVERSITY)

YENEPOYA UNIVERSITY

Project – High Level Design (HLD)

on

Natural Language Interface for E-Commerce Infrastructure

(ShopEase E-Commerce Chatbot)

Student Name: Muhammed Yasar KMC

Course: BCA

Guided by: Sindhu

Industry Mentor: Sumit KumarShukla

Project code : PRJN26-093

Registration no : 23bcaicd086

Table of Contents

1. Introduction

1.1 Scope of the Document

1.2 Intended Audience

1.3 System Overview

2. System Design

2.1 Application Design

2.2 Process Flow

2.3 Information Flow

2.4 Components Design

2.5 Key Design Considerations

2.6 API Catalogue

3. Data Design

3.1 Data Model

3.2 Data Access Mechanism

3.3 Data Retention Policies

3.4 Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements

7.1 Security Aspects

7.2 Performance Aspects

8. References

1. Introduction

Thisdocumentpresents the High Level Design (HLD) of the Natural Language Interface for E-Commerce Infrastructure — a rule-based chatbot system built using Python 3. The system allows users to interact with an e-commerce support platform using everyday conversational text, without requiring human customer service agents.

The chatbot handles common customer queries including order tracking, payment options, return policies, shipping information, and discount offers. It is available in two versions — a Command Line Interface (CLI) built in Python, and a Web Interface built with HTML, CSS, and JavaScript.

1.1 Scope of the Document

This HLD document covers the following:

- The overall system architecture and how all components connect
- The process flow from user input to bot response
- The data model used to store keyword-response pairs
- The interface design for both the CLI and web versions
- Non-functional requirements — security and performance

This document does NOT cover the detailed internal logic of individual functions. That is documented in the Low Level Design (LLD) document.

1.2 Intended Audience

Audience	Purpose
Professor / Evaluator	To assess the architecture and design choices made in the project
Developer (Student)	To use as a reference blueprint while building and extending the project
Industry Mentor	To review the technical approach and provide guidance
Future Developers	To understand the system before making enhancements

1.3 System Overview

The ShopEase Chatbot is a Natural Language Interface built on a rule-based keyword matching engine. It accepts free-form text from the user, identifies the intent using string matching, and returns a pre-written response from its knowledge base.

Property	Value
Project Title	Natural Language Interface for E-Commerce Infrastructure
Technology — Backend	Python 3 (chatbot.py, responses.py, utils.py)
Technology — Frontend	HTML5, CSS3, JavaScript (index.html)
Matching Technique	Basic String Matching using Python 'in' operator
Number of Topics Covered	7 (Orders, Payments, Returns, Shipping, Offers, Support, Cancellation)
External Libraries Used	None — Python standard library only
Development Tool	Visual Studio Code (VS Code)

2. System Design

TheShopEase Chatbotfollows a simple 3-layer architecture — Presentation, Logic, and Data — with no external dependencies or network calls required.

2.1 Application Design

The system is divided into three logical layers:

Layer	Description
Presentation Layer	Handles user input and displays output to the screen
Logic Layer	Keyword matching engine — finds the right reply for each input
Data Layer	Static in-memory knowledge base of keyword-response pairs

2.2 Process Flow

The following is the step-by-step flow for every user message sent to the chatbot:

1. User launches the chatbot by running: python chatbot.py in the terminal
2. The welcome banner is displayed on screen by display_banner() in utils.py
3. The program enters a continuous while True loop — waiting for user input
4. User types a message and presses Enter
5. The raw input is passed to get_response() in responses.py
6. Input is normalized to lowercase and extra spaces are removed using .lower().strip()
7. A nested loop checks each keyword in RESPONSES using the 'in' operator
8. If a keyword is found inside the input — the matching response is returned immediately
9. If no keyword matches after checking all entries — the FALLBACK message is returned
10. The response is formatted and displayed via format_bot_reply() in utils.py
11. The loop returns to step 3 and waits for the next message
12. When user types 'exit' or 'quit' — the loop breaks and the program closes

2.3 Information Flow

Information travels in a single direction — from user input through the logic layer to the screen output. The diagram below shows how data moves between components:

Terminal Version (Python):

From	To	Data Transferred
User Keyboard	chatbot.py	Raw text string entered by the user
chatbot.py	responses.py	Normalized (lowercase, stripped) input string
responses.py	chatbot.py	Matched response string or FALLBACK
chatbot.py	utils.py	Response string to be formatted
utils.py	Terminal Screen	Final formatted text printed to the screen

Web Version (JavaScript):

From	To	Data Transferred
HTML Input Box	sendMessage()	Raw text typed by the user
sendMessage()	getResponse()	Normalized input string
getResponse()	addMessage()	Matched response string
addMessage()	Browser Screen	Chat bubble rendered with reply text

2.4 Components Design

The project is made up of 4 files, each with a clearly defined single responsibility:

Component / File	Responsibility	
chatbot.py	Main entry point. Contains run_chatbot() — the while True input loop and command routing.	
responses.py	Knowledge base. Contains the RESPONSES list and get_response() matching function.	
utils.py	Display module. Contains display_banner(), display_help(), and format_bot_reply().	
index.html	Web version. A self-contained file with HTML structure, CSS styling, and JavaScript logic.	

Internal function summary:

Function	File	Purpose
run_chatbot()	chatbot.py	Starts and manages the continuous chat loop
get_response(input)	responses.py	Returns a matching response string for user input
display_banner()	utils.py	Prints the welcome header to the terminal
display_help()	utils.py	Prints the formatted command reference table
format_bot_reply(msg)	utils.py	Formats and prints the bot reply with indentation
sendMessage()	index.html	Handles the send button click on the web interface
getResponse(input)	index.html	JavaScript version of the keyword matching engine
addMessage(text,who)	index.html	Creates and adds a chat bubble to the screen

2.5 Key Design Considerations

Design Decision	Reason
Single HTML file for web version	Easy to share and open — no web server or installation needed
Static in-memory data — no database	Keeps the project lightweight and simple for a beginner project
3 separate Python files (modular design)	Each file has one job — easier to understand, test, and extend
while True loop in chatbot.py	Keeps the conversation running until the user explicitly exits
.lower().strip() normalization	Makes matching case-insensitive and tolerant of accidental spaces
First-match-wins strategy	Simple and fast — no ambiguity, predictable behaviour every time
textContent instead of innerHTML (JS)	Prevents XSS (Cross-Site Scripting) attacks in the web version

2.6 API Catalogue

This project does not consume any external APIs. All functions serve as internal APIs between the three Python modules. The table below lists each internal API:

Function (Internal API)	Input	Output
get_response(user_input)	String — raw user text	String — matched response or FALLBACK
display_banner()	None	None — prints to terminal
display_help()	None	None — prints command table
format_bot_reply(message)	String — response text	None — prints formatted reply
getResponse(userInput) [JS]	String — user text	String — matched response
addMessage(text, sender) [JS]	String, String	None — renders chat bubble

3. Data Design

Thesystemusesasingle in-memory data structure. No external database, file storage, or cloud service is used.

3.1 Data Model

All chatbot knowledge is stored in a Python list of dictionaries named RESPONSES, defined in responses.py. Each dictionary represents one topic category:

Field	Type
keywords	List of strings
response	String (multi-line)
FALLBACK	String constant

Example entry from the RESPONSES list:

```
{ "keywords": ["order status", "track order", "show status"],

  "response": "ORDER STATUS\nYour order #ORD-20481 is: OUT FOR DELIVERY..." }
```

3.2 Data Access Mechanism

Data is accessed using a sequential linear search — no indexing, querying, or sorting is used:

1. Iterate through RESPONSES list from index 0 to end
2. For each entry, check all keywords using Python's 'in' operator
3. Return the first matching entry's response immediately
4. If no match found after the full loop — return FALLBACK string

Property	Value	
Algorithm	Sequential Linear Search	
Time Complexity	O(R x K) — R = response entries (7), K = keywords per entry (~5)	
Space Complexity	O(1) — no extra memory used during the search	
Worst Case	35 checks (7 entries x 5 keywords) — completes in under 1ms	

3.3 Data Retention Policies

The current version does not retain any data after the session ends. All conversation data exists only in RAM during the active session and is permanently lost when the program closes. No log files, databases, or cookies are written.

Future improvement: Chat history could be saved to a .txt file using Python's built-in open() and write() functions, allowing conversations to be reviewed after the session.

3.4 Data Migration

Not applicable for the current version. The system uses hardcoded static data with no external storage. Future versions that connect to a real database (e.g., SQLite or MySQL) would require a migration plan to move the RESPONSES list into structured database tables.

4. Interfaces

The chatbot is available through two distinct interfaces — a terminal-based CLI and a browser-based web interface. Both use the same keyword matching logic.

4.1 Terminal Interface (CLI)

Interface Element	Description
Input Method	Python input() — waits for user keyboard text and Enter key
Output Method	Python print() — displays text in the terminal window
Welcome Screen	ASCII banner printed once on startup by display_banner()
Help Screen	Formatted 2-column command reference table by display_help()
Bot Reply	Indented, bordered text block with 0.3 second delay by format_bot_reply()
Exit Condition	User types exit, quit, or bye — loop breaks and program closes

4.2 Web Interface (Browser)

Interface Element	Description
Chat Window	420x620px white card, centered on screen with rounded corners
Header Bar	Dark green (#075e54) top bar with bot name and Online status
Quick Buttons	Pill-shaped shortcut buttons for Order Status, Payment, Offers, etc.
User Message Bubble	Green bubble (#dcf8c6) appearing on the right side of screen
Bot Message Bubble	White bubble appearing on the left side of screen
Typing Indicator	Three animated bouncing dots shown for 0.8 seconds before each reply
Input Bar	Text input at the bottom with a circular green send button
Enter Key Support	Pressing Enter triggers send — same as clicking the send button

5. State and Session Management

Thecurrentchatbot is completelystateless.Itdoesnot remember previous messages or maintain any session context between turns. Every user message is processed independently as a fresh query.

Aspect	Current Behaviour
Session Memory	None — each message is processed independently
User Identity	Not tracked — no login, username, or account system
Conversation History	Not stored — all chat data is lost when the session ends
Context Awareness	None — the bot cannot refer back to previous messages
Multi-turn Support	Not supported — each query is standalone

Example of stateless limitation: If a user asks 'show my order status' and then follows up with 'what is the tracking number?' — the bot cannot answer because it has no memory of the previous exchange.

Future improvement: Maintaining a conversation_history = [] list would allow the bot to store and reference previous exchanges within a session.

6. Caching

No caching mechanism is implemented in the current version. Since the RESPONSES list is small (7 entries, approximately 2KB total) and is already loaded into RAM the moment the program starts, caching provides no additional benefit.

Every keyword search completes in under 1 millisecond — making caching unnecessary at this scale.

Future improvement: If the system were extended to fetch responses from an external database or REST API, caching frequently requested responses (such as shipping info or payment options) would significantly reduce response time and server load.

7. Non-Functional Requirements

7.1 Security Aspects

Security Concern	How It Is Handled in This Project
Input Sanitization	User input is converted to lowercase and whitespace is stripped before any processing
No Personal Data Collection	The chatbot never asks for or stores names, passwords, card numbers, or any user data
No Network Exposure	The terminal version runs fully offline — zero network attack surface
XSS Prevention (Web Version)	JavaScript uses textContent instead of innerHTML — prevents script injection attacks
No File System Access	The program only reads from in-memory data — it cannot access or modify system files
No Third-Party Libraries	No external packages are used — eliminates supply chain security risks

7.2 Performance Aspects

Performance Metric	Value / Description
Response Time	Under 1ms — maximum 35 keyword checks (7 entries x 5 keywords)
Memory Usage	Less than 5MB — entire program and data fits comfortably in RAM
Startup Time	Under 1 second — no database connection, file loading, or network call required
Concurrent Users (CLI)	1 user — single-threaded terminal application
Concurrent Users (Web)	Unlimited — index.html is a static file served directly by the browser
Scalability Limit	Up to 100 response categories before performance impact needs review

8. References

- Python 3Official Documentation — https://docs.python.org/3/
- MDN Web Docs (HTML, CSS, JavaScript) — https://developer.mozilla.org/
- Visual Studio Code Documentation — https://code.visualstudio.com/docs
- IBM HLD Format Template — provided by institution
- Project Source Code — ecommerce_chatbot/ folder (submitted separately)